# UNIVERSAL PROMPT ENGINEERING GUIDELINES

A practical framework for accurate, efficient, reliable code generation

**Designed for:** GPT-5.6 Luna and Gemini 3.8 Flash

**Core objective: communicate the right information, establish clear boundaries, define measurable success, and make the result verifiable.**

## The six criteria for judging every prompt and solution

- **Prompt quality and structure:** clear problem, outcome, context, environment, requirements, restrictions, and output format.
- **Accuracy and relevance:** technically correct, directly responsive, compatible, and free from fabricated details.
- **Creativity:** practical improvements and alternatives that add value without overriding mandatory requirements.
- **Problem solving:** root-cause analysis, decomposition, dependencies, edge cases, and failure handling.
- **Efficiency:** appropriate algorithms, resource use, database access, dependencies, and maintainability.
- **Final output quality:** complete, usable files, working setup instructions, honest test results, and disclosed limitations.

**Judging question:** Can a competent developer understand the task without guessing, and can the user verify and use the result?

# 1. Fundamental principles of effective prompting

## Define the outcome before the process

Start with what must be achieved, why it is needed, who will use it, and what successful completion looks like. Give the AI freedom to choose an implementation approach unless a particular step is essential for correctness, safety, or compatibility.

## Provide sufficient, relevant context

- Project purpose, problem statement, target users, current development stage, and existing behaviour.
- Relevant source files, folder structure, screenshots, database schema, API documentation, logs, and error messages.
- Technical environment, dependencies, versions, deployment target, and known limitations.
- Previous decisions or constraints that affect the requested change.

Explain what each supplied file contains and why it matters. For large projects, provide the structure and relevant files before requesting implementation.

## Use a consistent structure

```
# ROLE
# PROJECT CONTEXT
# OBJECTIVE
# FUNCTIONAL REQUIREMENTS
# TECHNICAL REQUIREMENTS
# CONSTRAINTS
# EXPECTED BEHAVIOUR
# EDGE CASES
# SUCCESS CRITERIA
# VALIDATION
# OUTPUT FORMAT
```

Use Markdown headings or XML-style tags consistently. Do not combine several complicated organizational systems without a reason.

## Make instructions measurable

Replace vague terms such as “professional,” “advanced,” “perfect,” or “responsive” with observable requirements. Example: specify that pages must remain usable on phones, tablets, and desktops without horizontal overflow.

## Prioritize and resolve conflicts

1. Correctness and mandatory requirements.
2. Data integrity and security.
3. Functional completeness.
4. Compatibility with the existing environment.
5. Performance, maintainability, and code quality.
6. Visual polish and optional improvements.

**If two requirements conflict or an essential detail is missing, identify the issue and ask a focused clarification question instead of silently guessing.**

# 2. Define the software requirements precisely

## Describe each feature through four dimensions

- **Input:** what the user or system supplies, format, required fields, permitted values, and validation.
- **Processing:** calculations, business rules, conditions, dependencies, and transformations.
- **Output:** what must be displayed, stored, returned, or generated; define success and failure behaviour.
- **Interaction:** available user actions, resulting state changes, feedback, and interface behaviour.

## Specify the technical environment

State the details that determine whether generated code can run:

- **Language and runtime:** language, version, and relevant runtime.
- **Application stack:** frontend, backend, frameworks, database, and required libraries.
- **Environment:** operating system, IDE, deployment platform, and hosting constraints.
- **Integrations:** APIs, authentication methods, external services, and data formats.
- **Existing project:** folder structure, important files, and current conventions.

Do not let the AI assume a stack that has already been selected. If no stack is specified, ask for a recommendation based on actual requirements.

## Architecture should fit the project

For larger systems, describe the needed separation of frontend, backend, business logic, data access, routes, configuration, utilities, logging, error handling, and tests. Choose the simplest architecture suitable for the scale; do not introduce patterns merely to make a small project appear sophisticated.

## Use examples for non-obvious behaviour

Examples are especially valuable for input/output formats, database records, calculations, API responses, validation, sorting, ranking, and UI interactions. Include normal and unusual cases where they clarify the rule.

# 3. Data, integrations, interface, and constraints

## Database requirements

- Identify tables, columns, data types, primary and foreign keys, relationships, uniqueness, nullability, defaults, and indexes.
- Specify update/delete behaviour, validation, transactions, and migration requirements.
- For an existing database, inspect the actual schema before proposing changes.
- Use safe migrations and backups where appropriate; verify the result after a migration.

**DATA-INTEGRITY RULE: Never delete, overwrite, reset, truncate, or modify existing user data unless explicitly authorized. Preserve records and relationships.**

## API and integration requirements

Where relevant, provide the provider, documented endpoint, HTTP method, authentication, parameters, request body, response shape, error codes, rate limits, timeouts, retries, and data format. Supply official documentation when available.

Never invent endpoints, API keys, parameters, or response structures. Verify unfamiliar or potentially changed APIs against current official documentation when access is available.

## UI/UX requirements

Describe layout, navigation, page structure, typography, colours, spacing, components, forms, buttons, tables, charts, responsiveness, accessibility, and loading, empty, error, and success states. For screenshots, identify what must be reproduced and what may be improved.

## Separate hard constraints from preferences

- **Mandatory:** non-negotiable requirements that must be satisfied.
- **Restrictions:** prohibited technologies, operations, or changes.
- **Preferences:** desired approaches that may be adjusted with justification.
- Examples: do not remove features, alter API contracts, expose secrets, use cloud services, or rewrite unrelated files unless explicitly authorized.

# 4. Reliability, security, performance, and success

## Consider relevant edge cases

- **Inputs:** empty or missing fields, wrong types, invalid formats, extreme values, duplicates, and special characters.
- **Application:** no records, concurrent users, repeated submissions, network failures, database failures, API timeouts, expired sessions, and restarts.
- **Data:** null values, inconsistent relationships, invalid references, concurrent updates, and partial transactions.

Ask the AI to identify additional realistic edge cases. Do not force irrelevant cases into a small program.

## Security from the beginning

- Validate and safely handle inputs; use parameterized database queries.
- Use secure password hashing, authentication, authorization, and session protections where applicable.
- Address SQL injection, cross-site scripting, CSRF, unsafe uploads, and access-control risks as relevant.
- Protect secrets; do not hardcode production credentials or expose sensitive data.
- Use rate limiting and secure configuration where justified.

Explain significant risks and avoid claims that any application is completely secure.

## Specify workload and performance expectations

Provide approximate users, concurrent requests, database size, file limits, response-time expectations, memory/storage limits, and likely traffic growth. Evaluate query count, indexes, pagination, transactions, repeated work, and resource consumption. Optimize genuine bottlenecks without sacrificing correctness or maintainability.

## Define testable completion criteria

A task is complete only when required features work, existing behaviour and data are preserved, invalid input is handled, the application starts, relevant tests pass, and setup instructions and limitations are provided.

# 5. Required implementation workflow — understand, plan, implement

For substantial coding tasks, follow the workflow below. Keep it proportionate: a small change needs a short plan, not an elaborate process.

## Stage 1 — Understand

1. Read the complete problem statement and inspect relevant files.
2. Understand project structure, architecture, data flow, and existing functionality.
3. Check dependencies, versions, constraints, and risks.
4. Identify missing information that materially affects correctness.

Do not modify files before understanding the relevant implementation.

## Stage 2 — Plan

Briefly identify files to create or modify, reusable components, database changes, dependencies, important logic, integration points, likely failures, and validation methods.

## Stage 3 — Evaluate

- Does the approach satisfy every mandatory requirement?
- Is it compatible with the existing project and does it preserve behaviour?
- Is there a simpler reliable solution?
- Are dependencies, security risks, and data-integrity risks justified?

## Stage 4 — Implement

Use clear names, consistent formatting, suitable data structures, reusable functions, modularity where useful, meaningful error handling, and minimal necessary dependencies. Keep changes within scope.

**Do not over-engineer. Correctness, security, data preservation, and maintainability take priority over decorative complexity.**

# 6. Required implementation workflow — integrate, test, deliver

## Stage 5 — Integrate

Verify that components work together: frontend/backend requests, backend/database operations, API responses, forms, authentication, and displayed updates. Individually correct files do not automatically form a working application.

## Stage 6 — Test and verify

1. Syntax and import checks.
2. Dependency and configuration checks.
3. Unit and integration tests.
4. Database and input-validation tests.
5. Relevant edge-case and security checks.
6. Build, smoke, or end-to-end tests where feasible.

Use available tools and the actual project environment. Run the most relevant tests first; run the full suite when practical.

## Stage 7 — Fix and retest

1. Read the actual error message.
2. Trace the underlying cause to the relevant code.
3. Apply the smallest appropriate fix.
4. Check for side effects and rerun the failed test.
5. Run relevant regression tests.

Do not suppress errors or hide the underlying cause with an unreliable workaround.

## Stage 8 — Final review

Compare the implementation with the original requirements. Check missing features, calculations, imports, files, dependencies, database operations, security, UI consistency, setup instructions, and unintended changes.

**Declare success only for checks that were actually performed. Clearly identify anything not tested or verified.**

# 7. Special rules for existing projects

## Inspect before modifying

Never assume how an existing application works. Read relevant files, inspect dependencies, trace function calls and data flow, and understand the current behaviour before editing.

## Preserve by default

- Do not remove existing features or rename important functions unnecessarily.
- Do not change database structures, routes, API contracts, or user data without authorization.
- Do not redesign unrelated pages or rewrite the whole project when a targeted change is sufficient.
- Follow existing naming, folder, formatting, error-handling, database, API, and configuration conventions.

## Make minimal, controlled changes

Prefer a targeted fix. If multiple files must change, explain why each is necessary. Before adding a library, verify that it exists, is compatible, is needed, and is not already covered by an existing dependency.

## Protect data during database changes

- Understand the current schema and preserve existing records.
- Avoid destructive migrations; use transactions where appropriate.
- Back up important data before risky operations.
- Verify the resulting schema and records after migration.

## Report changed files

At completion, list every changed or created file, what changed and why, new dependencies, database modifications, and configuration changes. Include any required migration or deployment steps.

**A successful modification improves the requested area without silently damaging unrelated working functionality.**

# 8. Advanced prompting techniques

## Zero-shot prompting

Give a clear task without examples. Use it when requirements and expected output are conventional and straightforward.

## Few-shot prompting

Provide representative input/output examples when formatting, unusual business rules, or exact behaviour must be consistent. Include normal and boundary examples.

## Decomposition

Split complex work into meaningful parts such as authentication, database design, backend APIs, frontend, integration, testing, and deployment. Use decomposition when it reduces complexity or improves verification.

## Comparative reasoning

Ask the AI to compare alternatives against relevant criteria—such as concurrency, hosting, maintenance, scalability, and cost—then justify a recommendation. Do not request alternatives when the technology choice is already mandatory.

## Assumption management

**Do not invent important requirements. If missing information materially changes architecture, functionality, security, or correctness, ask a focused question. For minor details, make a reasonable assumption, state it briefly, and continue.**

## Self-verification and independent review

Ask the AI to compare the result against requirements and identify omissions. For critical work, request a separate review for logic errors, security vulnerabilities, data-loss risks, assumptions, performance, and edge cases. Self-review is useful but does not replace executing tests.

## Iterative refinement and traceability

Give specific feedback tied to observed behaviour. For complex work, map each requirement to its implementation and verification method so omissions are visible.

# 9. Model-specific prompting guidance

## GPT-5.6 Luna

Use outcome-first instructions, precise context, explicit constraints, approval boundaries, and measurable success criteria. State each instruction once; avoid unnecessary repetition. Allow efficient implementation choices within the constraints and request actual validation when tools are available.

**Practical reasoning-effort starting points from the source guide:** none/low for simple tasks; medium for general development; high for complex algorithms or multi-file work; xhigh/max for especially difficult architecture or debugging. These are starting points, not guaranteed rankings.

## Autonomy and approval boundaries

- When implementation is requested, allow in-scope file inspection, edits, and non-destructive tests.
- Require approval for destructive changes, external writes, purchases, or scope expansion.
- For explanation or review-only requests, do not implement changes.
- For authorized implementation, complete ordinary in-scope steps without repeatedly asking permission.

## Gemini 3.8 Flash

Use direct, concise, structured instructions. Separate context, task, constraints, and output. Define ambiguous terms, put essential requirements early, and provide context before the specific task for large codebases or documents.

Use examples for unusual behaviour and request verification of important decisions. The source guide lists low for simple tasks, medium for general coding, and high for complex reasoning or multi-step work; medium is a sensible starting point for substantial tasks.

## Reusable model-neutral instruction

```
Understand the intended result and inspect relevant context. Implement the simplest reliable
solution that satisfies every mandatory requirement. Preserve existing functionality and data.
Validate with available tools, fix defects, and retest. Never claim success without evidence;
disclose assumptions and unverified areas.
```

Model-specific guidance should not be mixed with unrelated modality instructions.

# 10. Mandatory cross-checking and error prevention

For substantial work, check each requirement independently. Code that appears related to a requirement is not proof that the requirement works.

## Requirement verification

- Is the requirement implemented correctly?
- Does it work with the other components?
- Was it tested, and is there evidence for the completion claim?

## Syntax and logic verification

Check syntax, brackets, indentation, imports, names, function calls, data types, strings, and configuration. Then verify conditions, loops, calculations, sorting, filtering, dates/times, boundaries, state transitions, duplicates, and data consistency.

For critical calculations, verify with independent examples or test cases.

## Integration verification

Confirm required files exist, imports resolve, functions are called correctly, frontend routes match backend endpoints, queries match the schema, API responses match expected formats, and data passes correctly between components.

## Security and regression verification

Check secrets, SQL injection, unsafe input, access controls, sensitive-data exposure, file operations, authentication, and error disclosure. When modifying existing software, confirm previously working functionality still works.

## Output verification

Confirm all requested files, names, locations, dependencies, configuration, setup instructions, and required code are included.

# 11. Efficiency rules and master prompt structure

## Keep prompts concise but complete

- Remove repeated instructions, unrelated background, duplicate constraints, irrelevant examples, and motivational filler.
- Ask only questions whose answers materially affect correctness or risk.
- Prefer targeted modifications over unnecessary rewrites.
- Use built-in features or existing libraries when appropriate; add dependencies only for meaningful value.
- Avoid unnecessary microservices, abstraction, design patterns, databases, caching, APIs, or complex deployment systems.
- Match the solution to workload, available resources, budget, and maintenance needs.

## Reusable master prompt template

```
# ROLE
Act as a senior software engineer and quality reviewer.

# PROJECT CONTEXT
Name, purpose, problem, users, development stage, and existing behaviour.

# OBJECTIVE
Exactly what must be built, fixed, modified, or optimized.

# ENVIRONMENT
Language/version, frontend, backend, database, libraries, OS, IDE, deployment.

# EXISTING PROJECT
Folder structure, relevant files, working features, known issues.

# FUNCTIONAL REQUIREMENTS
For each feature: input, processing, output, and user interaction.

# TECHNICAL REQUIREMENTS
Architecture, data model, APIs, data flow, validation, errors, integration, performance.

# CONSTRAINTS
Mandatory requirements, prohibited changes, and preferences.

# DATA / SECURITY
Preserve existing data; specify relevant protections and access rules.

# EDGE CASES
Invalid/missing input, duplicates, boundaries, failures, concurrency.

# VALIDATION / SUCCESS
Define measurable tests and conditions for completion.

# OUTPUT
Summary, changed files, complete code, setup, migrations, tests, limitations.
```

# 12. Specialized prompt patterns and quality scorecard

## Debug an existing program

Inspect relevant code, understand intended behaviour, trace the root cause, identify exact files/functions, apply the smallest reliable correction, preserve unrelated functionality, test the fix, and check regressions. Explain cause, changes, and validation. Do not claim a fix without verification.

## Add a feature

Inspect architecture, relevant files, dependencies, and interactions first. Implement the stated feature, preserve existing data and behaviour, modify only necessary files, avoid unrelated dependencies, and test both the new feature and affected existing functionality.

## Optimize code

Identify genuine bottlenecks in time/space complexity, queries, memory, network operations, repeated calculations, or dependencies. Preserve output and behaviour. Measure improvements where possible; otherwise explain the technical basis without claiming measured gains.

## Build a complete application

Understand the problem, architecture, data flow, interactions, and workload. Implement frontend, backend, database, and integrations; validate data, errors, security, responsiveness, installation, and execution. Do not describe a partial implementation as complete.

## Review without modifying

Inspect for functional defects, logic errors, security, performance, data integrity, compatibility, edge cases, and maintainability. For each issue give file/location, description, severity, impact, and recommended correction. Separate confirmed defects from potential risks.

## Quality scorecard

| Criterion | Weight |
|---|---|
| Prompt quality and structure | 20% |
| Accuracy and relevance | 20% |
| Creativity | 10% |
| Problem solving | 20% |
| Efficiency and optimization | 10% |
| Final output quality | 20% |

Score each criterion from 0–10 (0–2 very poor, 3–4 below expectations, 5–6 acceptable, 7–8 good, 9 excellent, 10 outstanding). Weighted score = sum of (criterion score × weight). A high total cannot excuse a critical failure such as data loss or failure of the central function.

# 13. Final submission checklist and golden rules

## Before sending a coding prompt

- Objective and problem are clear; sufficient relevant context is supplied.
- Mandatory features, environment, versions, constraints, and output format are specified.
- Existing files, functionality, database requirements, and expected behaviour are explained.
- Relevant edge cases, security needs, measurable success criteria, and conflicts are considered.

## Before accepting generated code

- The solution addresses the actual problem and every mandatory requirement.
- Syntax, logic, integration, input handling, and relevant edge cases have been checked.
- Existing functionality and data are preserved; security risks are addressed.
- Dependencies are justified; performance is appropriate; setup and execution instructions are complete.
- Test results are honest and limitations or unverified areas are disclosed.

## The 25 golden rules

1. Define the outcome before implementation.
2. Explain the problem, not just the desired code.
3. Provide sufficient project context.
4. Specify the exact programming environment.
5. Separate mandatory requirements from preferences.
6. Use a consistent structure.
7. Avoid ambiguity and contradictions.
8. Use examples for complicated behaviour.
9. Define expected inputs and outputs.
10. Specify database and API requirements.
11. Consider security from the beginning.
12. Consider realistic edge cases.
13. Define measurable success criteria.
14. Inspect existing code before modifying it.
15. Preserve existing functionality.
16. Protect existing data.
17. Avoid unnecessary rewrites and dependencies.
18. Prefer simple, maintainable solutions.
19. Validate with actual tools wherever possible.
20. Fix defects and retest.
21. Verify integration, not just individual files.
22. Never accept fabricated test results.
23. Cross-check against original requirements.
24. Disclose assumptions and limitations.
25. Deliver complete, usable output.

**FINAL PRINCIPLE: Understand → Plan → Implement → Integrate → Test → Verify → Deliver. A solution is complete only when its actual behaviour has been checked against the requirements—not merely when code generation has finished.**